実験データ記録API設計書

メタデータ

- **作成日**: 2026-01-27
- **バージョン**: 3.4.0
- **最終更新**: 2026-03-16
- **担当**: Development Team
- **実装状況**: ✅ 実装済み

概要

このドキュメントは、実験作業者が実験データを記録するためのAPI仕様を定義します。

主な機能

- 実験データー括保存（複数プレースのテンプレートデータ）
- DataGrid（2次元配列）対応
- ネストプロパティ対応
- All or Nothing方式のトランザクション保証

対象ユーザー

- 実験作業者（Worker）: 実験データ記録
- 実験管理者（Manager）: 上記 + データ確認、承認
- システム管理者（Admin）: 全権限

📋 API一覧

No	HTTPメソッド	エンドポイント	説明	実装状態	権限
1	POST	/api/v1/experiments/projects/:projectName/methods/:methodId/tasks/:taskNo/data	実験データー括保存	✅ 実装済み	experiment_worker のみ書込可
2	GET	/api/v1/experiments/projects/:projectName/methods/:methodId/tasks/:taskNo/data	テンプレート実績一覧取得	✅ 実装済み	experiment_worker 読取専用

---

1. 実験データー括保存API

概要

タスク内の複数プレースの実験データを一括で保存します。テンプレート（Material, Condition, Result）ごとにデータを記録でき、All or Nothing方式のトランザクション保証により、すべてのデータが正常に保存されるか、すべて失敗するかのいずれかになります。

エンドポイント

```
POST /api/v1/experiments/projects/:projectName/methods/:methodId/tasks/:taskNo/data
```

HTTPメソッド

POST

認証要件

- JWT認証必須
- ロール: experiment_worker のみ書込可
- テナント分離必須

ミドルウェアチェーン

```
router.post('/experiments/projects/:projectName/methods/:methodId/tasks/:taskNo/data',
  auth,                          // JWT認証
  tenantIsolation,               // テナント分離
  permissionCheck(['experiment_worker']), // 権限チェック
  experimentDataController.saveBatchData
);
```

リクエスト

パスパラメータ

パラメータ	型	必須	説明
projectName	string	○	プロジェクト名
methodId	string	○	メソッドID
taskNo	integer	○	タスク連番

リクエストボディ

> ***注意***: `data` フィールドは `GET /api/v1/experiments/.../data` レスポンスの `templates[].data` と同一形式（`PropertySet` 形式）です。
> フロントエンドは GET で取得した `data` を、ユーザーが修正した値（`properties[].value`）のみ書き換えてそのまま POST してください。

```
{
  "records": [
    {
      "placeId": "place_001",
      "data": {
        "templateId": "mat_001",
        "templateType": "material",
        "properties": [
          { "key": "temperature", "xsiType": "floatType", "value": 25.5, "units": "°C" },
          { "key": "pressure",    "xsiType": "floatType", "value": 1013,  "units": "hPa" }
        ]
```

```
      },
      "comment": "材料データ記録完了"
    },
    {
      "placeId": "place_002",
      "data": {
        "templateId": "cond_001",
        "templateType": "condition",
        "properties": [
          { "key": "humidity", "xsiType": "floatType", "value": 60, "units": "%" }
        ]
      },
      "comment": "条件データ記録中"
    }
  ]
}
```

リクエストボディのパラメータ詳細

パラメータ	型	必須	説明	制約
records	array	○	実験データ配列	1件以上の配列

records配列の各要素

パラメータ	型	必須	説明	制約
placeId	string	○	プレースID	
data	object	○	実験データ（PropertySet形式）	`{ templateId, templateType, properties }` の形式
data.templateId	string	○	テンプレートID	
data.templateType	string	○	テンプレート種別	material, condition, result
data.properties	array	○	プロパティ配列	`[{ key, xsiType, value, units, description?, formatString?, scaleFactor? }]` の形式

| comment | string | × | コメント | 最大2000文字 |

> *注意: `templateType` ・ `templateId` はトップレベルには含めず、`data` オブジェクト内に含めてください。*

レスポンス

成功時（201 Created）

```
{
  "success": true,
  "data": {
    "saved": 2,
    "records": [
      {
        "id": "770e8400-e29b-41d4-a716-446655440002",
        "projectId": "550e8400-e29b-41d4-a716-446655440000",
        "methodId": "method_001",
        "taskNo": 1,
        "placeId": "place_001",
        "templateType": "material",
        "templateId": "mat_001",
        "data": {
```

```
          "temperature": {
            "value": 25.5,
            "unit": "°C"
          },
          "pressure": {
            "value": 1013,
            "unit": "hPa"
          }
        },
        "status": "completed",
        "comment": "材料データ記録完了",
        "recordedBy": "880e8400-e29b-41d4-a716-446655440003",
        "recordedAt": "2026-01-27T10:30:00Z",
        "organizationId": "org_maiml_lab_001",
        "createdAt": "2026-01-27T10:30:00Z",
        "updatedAt": "2026-01-27T10:30:00Z"
      },
      {
        "id": "880e8400-e29b-41d4-a716-446655440003",
        "projectId": "550e8400-e29b-41d4-a716-446655440000",
        "methodId": "method_001",
        "taskNo": 1,
        "placeId": "place_002",
        "templateType": "condition",
        "templateId": "cond_001",
        "data": {
          "humidity": {
            "value": 60,
            "unit": "%"
          }
        },
        "status": "running",
        "comment": "条件データ記録中",
        "recordedBy": "880e8400-e29b-41d4-a716-446655440003",
        "recordedAt": "2026-01-27T10:30:00Z",
        "organizationId": "org_maiml_lab_001",
        "createdAt": "2026-01-27T10:30:00Z",
        "updatedAt": "2026-01-27T10:30:00Z"
      }
    ]
  },
  "meta": {
    "timestamp": "2026-01-27T10:30:00Z",
    "requestId": "req_abc123"
  }
}
```


エラー時（400 Bad Request）


```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Validation failed for record 1",
    "details": [
      {
        "index": 0,
        "field": "placeId",
        "message": "Place not found",
        "code": "RESOURCE_NOT_FOUND"
      }
    ]
  },
  "meta": {
    "timestamp": "2026-01-27T10:30:00Z",
    "requestId": "req_abc123"
```

```
  }
}
```

エラー時（403 Forbidden）

```
{
  "success": false,
  "error": {
    "code": "PERMISSION_INSUFFICIENT",
    "message": "Project is not in editable status"
  },
  "meta": {
    "timestamp": "2026-01-27T10:30:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時（404 Not Found）

```
{
  "success": false,
  "error": {
    "code": "RESOURCE_NOT_FOUND",
    "message": "Task not found or does not belong to your organization"
  },
  "meta": {
    "timestamp": "2026-01-27T10:30:00Z",
    "requestId": "req_abc123"
  }
}
```

ビジネスロジック

1. **プロジェクトステータス確認**
   - プロジェクトステータスが編集可能（**project_status = 2（タスク実行中）のみ**）であることを確認
   - project_status が 2 以外の場合は `ProjectNotEditableError` をスローして 403 エラーを返す
2. **タスク存在確認**
   - taskNoが指定されたproject_id、method_idに属することを確認
   - organization_idでマルチテナント分離
3. **リクエストバリデーション**
   - records配列の各要素をバリデーション
   - 1件でもエラーがあれば全体を拒否（All or Nothing）
4. **実験データー括保存（ループ処理）**
   - records配列の各要素に対して処理
   - template_dataテーブルにUPSERT（INSERT or UPDATE）
   - 一意キー: (organization_id, project_id, method_id, task_no, place_id)
   - 既存レコードがある場合、template_type、template_id、dataなどを上書き

- dataフィールドにJSONB形式で保存
- recorded_byに現在のユーザーIDを設定

5. **みなし進捗の自動更新**（自組織の `use_deemed_progress = true` の場合のみ）
   - MAIMLファイルをダウンロード＆パースしてアーク情報を取得
   - 保存されたプレースIDからアーク経由でトランジションIDを解決
   - 各トランジションについて以下を実行:
     - そのトランジションに接続された全プレース数（MAIML由来）を総数とする
     - `template_data` テーブルから実績登録済みプレース数をカウント
     - ステータスを決定:
       - 実績0件 → 更新しない
       - 実績1件以上かつ全件未満 → `IN_PROGRESS` (3)
       - 全件登録済み → `COMPLETED` (5)
     - 進捗率を計算: `Math.round(実績登録済み数 / 総数 × 100)`
     - `transition_data` テーブルの status / progress_rate を更新（状態遷移バリデーションなし）
   - MAIMLファイルが存在しない等のエラー時はスキップ（メイン処理には影響しない）

6. **トランザクション保証（All or Nothing）**
   - すべてのINSERT/UPDATEを1トランザクションで実行
   - 1件でも失敗したら全体をロールバック
   - すべて成功した場合のみコミット

バリデーション

リクエスト全体のチェック

- [ ] projectName は文字列
- [ ] methodId は文字列
- [ ] taskNo は1以上の整数
- [ ] records は配列で1件以上

records配列の各要素のチェック

- [ ] placeId は必須
- [ ] data はオブジェクト形式（必須）
- [ ] data.templateId は必須（文字列、1〜255文字）
- [ ] data.templateType は "material", "condition", "result" のいずれか（必須）
- [ ] data.properties は配列形式（必須）
- [ ] comment は2000文字以内（指定時）

関連性チェック

- [ ] projectNameは自組織に属するプロジェクト
- [ ] taskNoは指定されたprojectNameに属するタスク

- ☐ placeIdは指定されたtaskNoに属するプレース（オプション）
- ☐ templateIdは指定されたtemplateTypeに対応するテンプレート（オプション）

プロジェクトステータスチェック

- ☐ プロジェクトステータスが編集可能（draft/active/waiting_approval）

データベーステーブル

使用テーブル

1. **template_data**（主テーブル）
   - id, organization_id, project_id, method_id, task_no, place_id
   - template_type, template_id, data (JSONB)
   - status, comment
   - recorded_by, recorded_at
   - created_at, updated_at
   - **一意キー**: (organization_id, project_id, method_id, task_no, place_id)
2. **projects**（ステータス確認）
   - id, status, organization_id
3. **project_task**（存在確認）
   - project_id, task_no, organization_id

SQL例（一括保存）

```
-- トランザクション開始
BEGIN;

-- 1. プロジェクトステータス確認（マルチテナント分離）
SELECT status FROM projects
WHERE id = $1 AND organization_id = $2
AND status IN ('draft', 'active', 'waiting_approval');

-- 2. タスク存在確認
SELECT task_no FROM project_task
WHERE project_id = $1
  AND task_no = $3
  AND organization_id = $2;

-- 3. records配列の各要素に対してUPSERT（ループ処理）
-- (record 1)
INSERT INTO template_data (
  id, organization_id, project_id, method_id, task_no, place_id,
  template_type, template_id, data,
  status, comment,
  recorded_by, recorded_at,
  created_at, updated_at
) VALUES (
  gen_random_uuid(), $2, $1, $4, $3, $5,
  $6, $7, $8::jsonb,
  $9, $10,
  $11, NOW(),
```

```
  NOW(), NOW()
)
ON CONFLICT (organization_id, project_id, method_id, task_no, place_id)
DO UPDATE SET
  template_type = EXCLUDED.template_type,
  template_id = EXCLUDED.template_id,
  data = EXCLUDED.data,
  status = EXCLUDED.status,
  comment = EXCLUDED.comment,
  recorded_by = EXCLUDED.recorded_by,
  recorded_at = NOW(),
  updated_at = NOW()
RETURNING *;

-- (record 2)
INSERT INTO template_data (
  id, organization_id, project_id, method_id, task_no, place_id,
  template_type, template_id, data,
  status, comment,
  recorded_by, recorded_at,
  created_at, updated_at
) VALUES (
  gen_random_uuid(), $2, $1, $4, $3, $12,
  $13, $14, $15::jsonb,
  $16, $17,
  $11, NOW(),
  NOW(), NOW()
)
ON CONFLICT (organization_id, project_id, method_id, task_no, place_id)
DO UPDATE SET
  template_type = EXCLUDED.template_type,
  template_id = EXCLUDED.template_id,
  data = EXCLUDED.data,
  status = EXCLUDED.status,
  comment = EXCLUDED.comment,
  recorded_by = EXCLUDED.recorded_by,
  recorded_at = NOW(),
  updated_at = NOW()
RETURNING *;

-- (以降、records配列の要素数だけ繰り返し)

COMMIT;
```

パラメータ説明

```
$1: project_id (UUID)
$2: organization_id (string)
$3: task_no (integer)
$4: method_id (string)
$5: place_id (string) - record 1
$6: template_type (string) - record 1
$7: template_id (string) - record 1
$8: data (JSONB) - record 1
$9: status (string) - record 1
$10: comment (string) - record 1
$11: recorded_by (UUID) - 全レコード共通
$12: place_id (string) - record 2
$13: template_type (string) - record 2
... (以降、record数に応じて続く)
```

トランザクションの重要性

- **All or Nothing**: すべてのINSERT/UPDATEが成功するか、すべて失敗するかのいずれか
- **整合性保証**: 1件でも失敗したら全体をロールバック

- **ロック**: プロジェクトステータスチェックからコミットまで一貫性を保証

セキュリティ

マルチテナント分離

- [ ] organization_id フィルタ必須
- [ ] プロジェクトの所属組織を検証
- [ ] タスクの所属組織を検証

認証・認可

- [ ] JWT認証必須
- [ ] experiment_worker のみ書込可
- [ ] 編集可能なプロジェクトステータスのみデータ保存可能

入力検証

- [ ] SQLインジェクション対策（Parameterized Queries）
- [ ] JSONB型の適切な検証
- [ ] XSS対策（入力サニタイゼーション）

パフォーマンス

- インデックス: template_data (organization_id, project_id, method_id, task_no, place_id) - 一意キー（主キー）
- インデックス: template_data (organization_id, recorded_at) - 時系列検索用
- インデックス: template_data (project_id, task_no) - タスク単位の検索用
- JSONB型: GINインデックスを使用（data列）
- トランザクション: 必須（All or Nothing保証）
- バッチ処理: records配列のサイズに応じて複数のINSERTを実行
- キャッシュ: 不要（リアルタイム性重視）

---

2. テンプレート実績一覧取得API

概要

トランジションIDに隣接するプレースの実験実績データを取得します。
`DataRecordDialog` を開いた際に既存の入力済みデータを復元するために使用します。

処理は以下の2段階で行います：

1. `template_data` テーブルから対象タスクのレコードを全件取得
2. S3/Swift の MaiML ファイルをパースしてペトリネット構造を解析し、指定トランジションに隣接するプレースIDでフィルタリング

エンドポイント

```
GET /api/v1/experiments/projects/:projectName/methods/:methodId/tasks/:taskNo/data
```

HTTPメソッド

GET

認証要件

- JWT認証必須
- ロール: experiment_worker 読取専用
- テナント分離必須

ミドルウェアチェーン

```
router.get('/experiments/projects/:projectName/methods/:methodId/tasks/:taskNo/data',
  auth,                                  // JWT認証
  tenantIsolation,                       // テナント分離
  checkPermission('050000', 0),          // experiment_worker 読取専用
  [ query('transitionId').notEmpty().isString().isLength({ min: 1, max: 255 }) ],
  handleValidationErrors,
  experimentDataController.getTemplateData
);
```

リクエスト

パスパラメータ

パラメータ	型	必須	説明	制約
projectName	string	○	プロジェクト名	1〜255文字
methodId	string	○	メソッドID	1〜255文字
taskNo	integer	○	タスク連番	1以上の整数

クエリパラメータ

パラメータ	型	必須	説明	制約
transitionId	string	○	トランジションID	1〜255文字

リクエスト例

```
GET /api/v1/experiments/projects/sample_project/methods/method_001/tasks/1/data?transitionId=t1
```

処理フロー

1. パスパラメータ・クエリパラメータのバリデーション
2. JWT から organizationId を取得（tenantIsolation ミドルウェア）
3. `projects` テーブルから projectName → project_id を解決（organization_id でフィルタ）

4. `template_data` テーブルから `organization_id` + `project_id` + `method_id` + `task_no` が一致するレコードを全件取得
5. S3/Swift から MaiML ファイルをダウンロード（パス: `projects/{projectName}/{projectName}.maiml`）
6. `parsePetriNetStructure()` でペトリネット構造を解析
7. 指定 `transitionId` の存在確認（存在しない場合 404）
8. arcs から隣接プレースIDを特定（入力プレース・出力プレースの両方）
   - `arc.source === transitionId` → `arc.target`（出力プレース）
   - `arc.target === transitionId` → `arc.source`（入力プレース）
9. Step 4 で取得したレコードを隣接プレースIDでフィルタリング
10. フィルタリング後のデータ件数を確認
11. データが1件以上ある場合 → そのままレスポンスを返却（200 OK）
12. データが0件の場合 → MAIMLファイルのテンプレート定義からフォールバックレスポンスを生成して返却（200 OK）
    - `parser.getMethods()` で対象メソッドを特定
    - `materialTemplates` / `conditionTemplates` / `resultTemplates` をループ
    - 各テンプレートの `placeRefs` から隣接プレースIDに一致するものを抽出
    - `properties` を `{ key, xsiType, value: p.value ?? null, units }` 形式でマッピング（MAIMLに初期値がある場合はその値、なければ `null`）
    - `data` に `{ templateId, templateType, properties }` をセット（`taskRegistrationService.ts` の `property_set` と同形式）
    - `recordedBy` / `recordedAt` は `null` を設定
    - **注意**: テンプレートはメソッド直下（`method.materialTemplates` 等）と `method.program[]` 配下の両方を統合して検索する

レスポンス

成功時（200 OK）

```
{
  "success": true,
  "data": {
    "projectName": "sample_project",
    "methodId": "method_001",
    "taskNo": 1,
    "transitionId": "t1",
    "adjacentPlaceIds": ["p1", "p2"],
    "templates": [
      {
        "placeId": "p1",
        "templateType": "material",
        "templateId": "template-mat-001",
        "data": { "key": "value" },
        "recordedBy": "880e8400-e29b-41d4-a716-446655440003",
        "recordedAt": "2026-02-25T10:00:00.000Z"
      },
      {
        "placeId": "p2",
        "templateType": "condition",
        "templateId": "template-cond-001",
        "data": { "temperature": 25 },
```

```
        "recordedBy": "880e8400-e29b-41d4-a716-446655440003",
        "recordedAt": "2026-02-25T11:00:00.000Z"
      }
    ],
    "totalCount": 2
  },
  "meta": {
    "timestamp": "2026-02-25T17:00:00.000Z",
    "requestId": "req_abc123"
  }
}
```


隣接プレースの実績データが 0 件の場合（MAIMLテンプレートからフォールバック生成、200 OK）


```
{
  "success": true,
  "data": {
    "projectName": "sample_project",
    "methodId": "method_001",
    "taskNo": 1,
    "transitionId": "t1",
    "adjacentPlaceIds": ["p1", "p2"],
    "templates": [
      {
        "placeId": "p1",
        "templateType": "material",
        "templateId": "template-mat-001",
        "data": {
          "templateId": "template-mat-001",
          "templateType": "material",
          "properties": [
            { "key": "temperature", "xsiType": "stringType", "value": null, "units": "°C" },
            { "key": "pressure",    "xsiType": "floatType",  "value": null, "units": "hPa" }
          ]
        },
        "recordedBy": null,
        "recordedAt": null
      },
      {
        "placeId": "p2",
        "templateType": "condition",
        "templateId": "template-cond-001",
        "data": {
          "templateId": "template-cond-001",
          "templateType": "condition",
          "properties": [
            { "key": "humidity", "xsiType": "floatType", "value": null, "units": "%" }
          ]
        },
        "recordedBy": null,
        "recordedAt": null
      }
    ],
    "totalCount": 2
  },
  "meta": {
    "timestamp": "2026-02-25T17:00:00.000Z",
    "requestId": "req_abc123"
  }
}
```


> **注意**: `data` フィールドは `taskRegistrationService.ts` の `deleteAndInsertTemplateData` が `template_data.property_set` に格納する形式と同一です。`recordedBy` / `recordedAt` は未記録を表す `null` が設定されます。

エラーレスポンス

HTTPステータス	エラーコード	説明
400	VALIDATION_ERROR	パラメータ不正（transitionId 未指定など）
404	PROJECT_NOT_FOUND	指定したプロジェクトが存在しない
404	MAIML_NOT_FOUND	MaiML ファイルが S3/Swift に存在しない
404	TRANSITION_NOT_FOUND	MaiML 内に指定トランジションIDが存在しない
500	MAIML_PARSE_FAILED	MaiML のパース処理に失敗
500	DATABASE_ERROR	DB エラー

```
{
  "success": false,
  "error": {
    "code": "TRANSITION_NOT_FOUND",
    "message": "Transition 't999' not found in MaiML method 'method_001'"
  },
  "meta": {
    "timestamp": "2026-02-25T17:00:00.000Z",
    "requestId": "req_abc123"
  }
}
```

レスポンスフィールド詳細

data オブジェクト

フィールド	型	説明
projectName	string	リクエストのプロジェクト名
methodId	string	リクエストのメソッドID
taskNo	integer	リクエストのタスク連番
transitionId	string	リクエストのトランジションID
adjacentPlaceIds	string[]	MaiML から特定した隣接プレースIDの一覧
templates	array	実績データ配列（0件の場合は空配列）
totalCount	integer	templates 配列の件数

templates 配列の各要素

フィールド	型	説明
placeId	string	プレースID（テンプレートとプレースの紐付けに使用）
templateType	string	テンプレート種別（material / condition / result）
templateId	string	テンプレートID（テンプレート定義の特定に使用）
data	object	実験データ本体。DBに実績がある場合は `template_data.property_set` の値（ `PropertySet` 型: `{ templateId, templateType, properties: [{ key, xsiType, value, units }] }` 形式）。該当プレースがフォールバック時は `{ templateId, templateType,`

		`properties: [{ key, xsiType, value, units }]` 形式のMAIML定義値（未入力 `value` は `null` または MAIMLの初期値）
recordedBy	string | null	記録者のユーザーID（UUID）。`template_data.update_user_id` の値。DBに実績がない場合は `null`
recordedAt	string | null	記録日時（ISO 8601 形式）。`template_data.update_at` の値。DBに実績がない場合は `null`

バリデーション

- [ ] projectName は文字列（1〜255文字）
- [ ] methodId は文字列（1〜255文字）
- [ ] taskNo は1以上の整数
- [ ] transitionId は文字列（1〜255文字）、必須

データベーステーブル

使用テーブル

1. **projects**（project_id の解決）
   - project_name, id, organization_id
2. **template_data**（実績データ取得）
   - organization_id, project_id, method_id, task_no, place_id
   - template_type, template_id, data (JSONB)
   - recorded_by, recorded_at

SQL例（実績データ取得）

```
-- 1. project_id の解決
SELECT id FROM projects
WHERE project_name = $1
  AND organization_id = $2;

-- 2. 対象タスクの template_data を全件取得（プレースIDフィルタはアプリ層で実施）
SELECT
  place_id,
  template_type,
  template_id,
  data,
  recorded_by,
  recorded_at
FROM template_data
WHERE organization_id = $1
  AND project_id      = $2
  AND method_id       = $3
  AND task_no         = $4
ORDER BY place_id ASC;
```

セキュリティ

マルチテナント分離

- [ ] organization_id フィルタ必須（projects テーブル・template_data テーブル両方）
- [ ] クロステナントアクセス不可

認証・認可

- [ ] JWT認証必須
- [ ] experiment_worker 読取専用

入力検証

- [ ] SQLインジェクション対策（Parameterized Queries）
- [ ] パスパラメータの長さ制限

実装要件

フォールバック処理（0件時のMAIMLテンプレート生成）

フィルタリング後のデータが0件の場合、`MaimlParserTyped.getMethods()` を使用してMAIMLから初期値を生成する。
`taskRegistrationService.ts` の `deleteAndInsertTemplateData` と同一のロジックを使用すること。

```
/**
 * 0件時フォールバック: MAIMLテンプレート定義からレスポンスを生成
 *
 * @param parser - MaimlParserTyped インスタンス（MAIMLファイルは処理フロー Step 5 で取得済み）
 * @param methodId - メソッドID
 * @param adjacentPlaceIds - 隣接プレースIDの配列（処理フロー Step 8 で特定済み）
 * @returns フォールバック用 templates 配列
 */
function buildFallbackTemplates(
  parser: MaimlParserTyped,
  methodId: string,
  adjacentPlaceIds: string[]
): TemplateDataItem[] {
  const methods = parser.getMethods();
  const method = methods.find(m => m.id === methodId);
  if (!method) {
    throw new NotFoundError(`Method '${methodId}' not found in MaiML`);
  }

  const templateCategories: Array<{
    templates: typeof method.materialTemplates;
    templateType: 'material' | 'condition' | 'result';
  }> = [
    { templates: method.materialTemplates, templateType: 'material' },
    { templates: method.conditionTemplates, templateType: 'condition' },
    { templates: method.resultTemplates,    templateType: 'result' },
  ];

  const fallbackTemplates: TemplateDataItem[] = [];

  for (const { templates, templateType } of templateCategories) {
    if (!templates) continue;

    // 単一要素・配列の両方に対応（taskRegistrationService.ts と同一パターン）
    const templateArray = Array.isArray(templates) ? templates : [templates];

    for (const template of templateArray) {
      if (!template.placeRefs || template.placeRefs.length === 0) continue;
```

```
      // このテンプレートのプロパティ定義を構築（value は null で初期化）
      const properties = (template.properties || []).map(p => ({
        key: p.key || '',
        xsiType: p.xsiType || 'stringType',
        value: null,
        units: p.units || null,
      }));

      const data = {
        templateId: template.id || '',
        templateType,
        properties,
      };

      // 各 placeRef ごとに1エントリ生成（隣接プレースIDに一致するもののみ）
      for (const placeRef of template.placeRefs) {
        const placeId = placeRef.ref || '';
        if (!placeId) continue;
        if (!adjacentPlaceIds.includes(placeId)) continue; // 隣接プレースのみ

        fallbackTemplates.push({
          placeId,
          templateType,
          templateId: template.id || '',
          data,
          recordedBy: null,
          recordedAt: null,
        });
      }
    }
  }

  return fallbackTemplates;
}
```

使用箇所（サービス層）:

```
// Step 9 の後
const filteredRecords = allRecords.filter(r => adjacentPlaceIds.includes(r.place_id));

// Step 10-12: 件数確認とフォールバック処理
let templates: TemplateDataItem[];
if (filteredRecords.length > 0) {
  // Step 11: DB実績データをそのまま返却
  templates = filteredRecords.map(r => ({
    placeId: r.place_id,
    templateType: r.template_type,
    templateId: r.template_id,
    data: r.data,
    recordedBy: r.recorded_by,
    recordedAt: r.recorded_at,
  }));
} else {
  // Step 12: MAIMLテンプレート定義からフォールバック生成
  templates = buildFallbackTemplates(parser, methodId, adjacentPlaceIds);
}
```

パフォーマンス

- インデックス: `template_data (organization_id, project_id, method_id, task_no)` を使用
- MaiML パース: S3/Swift からのダウンロードを最小化（1回のみ、フォールバック時も同一インスタンスを再利用）
- キャッシュ: 不要（リアルタイム性重視）

🔗 関連ドキュメント

API設計書

- api_list.md - API一覧対応表
- api_project_management.md - プロジェクト管理API
- api_progress_monitoring.md - 進捗確認API（トランジションステータス更新APIを含む）
- api_issue_management.md - 問題管理API

規約・ガイドライン

- API設計ポリシー - API設計の統一ルール
- バックエンド規約 - 実装規約
- セキュリティ規約 - セキュリティ要件

参照ドキュメント

- データベース構造 - DB設計
- API構成定義 - API構成の参照ルール

📊 実装ファイルパス

ルートファイル

```
backend/src/routes/experimentRoutes.ts
```

コントローラーファイル

```
backend/src/controllers/experimentController.ts
```

サービスファイル

```
backend/src/services/experimentDataService.ts
```

テストファイル

```
backend/tests/routes/experimentRoutes.test.ts
backend/tests/services/experimentDataService.test.ts
backend/tests/controllers/experimentController.test.ts
```

✅ セキュリティチェックリスト

実装時に以下を確認してください：

認証・認可

- [ ] すべてのエンドポイントでJWT認証が必須
- [ ] tenantIsolationミドルウェアが適用されている
- [ ] permissionCheckで適切な権限が設定されている
- [ ] experiment_worker のみ書込可

マルチテナント分離

- [ ] すべてのDBクエリにorganization_idフィルタが含まれる
- [ ] クロステナントアクセスが不可能
- [ ] JOINするすべてのテーブルでorganization_idを検証

プロジェクトステータス確認

- [ ] 編集可能なプロジェクトステータスのみ保存可能
- [ ] archived/completedプロジェクトは編集不可

入力検証

- [ ] すべてのパラメータがバリデーションされている
- [ ] Parameterized Queriesを使用（SQLインジェクション対策）
- [ ] JSONB型の適切な検証
- [ ] XSS対策（入力サニタイゼーション）
- [ ] 文字列長制限が強制されている

トランザクション

- [ ] 実験データ保存とステータス更新を1トランザクションで実行
- [ ] ロールバック時の整合性を保証

エラーハンドリング

- [ ] 統一されたエラーレスポンス形式
- [ ] エラーメッセージに機密情報が含まれない
- [ ] 適切なHTTPステータスコードを返す

📝 変更履歴

バージョン	日付	変更内容	担当者

3.4.0	2026-03-16	maimlDataWriter仕様修正に伴うプロパティ形式の更新	Cline
		- `data.properties` 要素に `description?` ・ `formatString?` ・ `scaleFactor?` フィールドを追加	
		- `templateDataRepository.ts` の `PropertySetProperty` を廃止し `Property[]`（maiml-unified）に統一	
		- `taskRegistrationService.ts` でプロパティを `{ ...p, value: null }` として保存するよう変更（全フィールド保持）	
		- `eventLogRepository.ts` の xsi:type 文字列を `xs:string` → `stringType` 等に修正	
3.4.0	2026-03-12	ソースコード実装に合わせて仕様書を修正	Cline
		- 実装状況を「未実装」→「実装済み」に更新	
		- 実装ファイルパスを `experimentDataController.ts` → `experimentController.ts` 、 `experimentDataRoutes.ts` → `experimentRoutes.ts` に修正	
		- ミドルウェアチェーンのコントローラー関数名を `saveBatch` → `saveBatchData` に修正	
		- POST APIのプロジェクト編集可能ステータスを `draft/active/waiting_approval` → `project_status = 2（タスク実行中）のみ` に修正	
		- GET APIの `templates[].data` フィールド説明を `PropertySet` 型に合わせて修正	
		- GET APIの `recordedBy` / `recordedAt` を `update_user_id` / `update_at` カラムに対応させるよう修正	
		- フォールバック処理の `properties[].value` を `p.value ?? null` に修正（MAIML初期値対応）	
		- フォールバック処理のテンプレート取得元にメソッド直下と `method.program[]` 配下を統合する旨を追記	
3.3.0	2026-03-02	実験データ一括保存API みなし進捗対応	Cline

| | | - ビジネスロジック #5 にみなし進捗自動更新仕様を追加 | | | - `use_deemed_progress = true` の組織でデータ保存時に `transition_data` を自動更新 | | | - 実績1件以上: IN_PROGRESS (3)、全件完了: COMPLETED (5) | | | - progress_rate = Math.round(実績登録済み数 / 総プレース数 × 100) | | | - 状態遷移バリデーションなし、MAIMLエラー時はスキップ | | 3.2.0 | 2026-02-27 | テンプレート実績一覧取得API 0件時フォールバック仕様追加 | Cline | | | - フィルタリング後にデータ0件の場合、空配列ではなくMAIMLテンプレート定義からレスポンスを生成 | | | - 処理フローにStep 10〜12を追加（件数確認・フォールバック処理） | | | - 0件時の `data` は `taskRegistrationService.ts` の `property_set` と同形式（ `{ templateId, templateType, properties: [{ key, xsiType, value: null, units }] }` ） | | | - `recordedBy` / `recordedAt` は未記録時 `null` を返すよう仕様変更 | | | | - 実装要件セクションに `buildFallbackTemplates()` のコード例を追記 | | 3.1.0 | 2026-02-18 | エンドポイントと権限の仕様変更 | Cline | | | - エンドポイント: `:projectId` → `:projectName` に変更 | | | - 権限: `Worker以上` → `experiment_worker のみ書込可` に変更 | | | - パスパラメータ、バリデーション、セキュリティセクションを修正 | | 3.0.0 | 2026-02-18 | API構成変更（1エンドポイント化） | Cline | | | - API #2（トランジションステータス更新API）を削除 | | | - 進捗確認API（api_progress_monitoring.md）に移動 | | | - 実験データ記録APIは実験データ一括保存のみに専念 | | 2.1.0 | 2026-02-17 | エンドポイントパラメータ変更 | Cline | | | - API 1: projectId, methodId, taskNoをパスパラメータ化 | | | - API 1: エンドポイント変更 `/experiments/data` → `/experiments/projects/:projectId/methods/:methodId/tasks/:taskNo/data` | | | - API 2: taskNoをパスパラメータ化 | | | - API 2: エンドポイント変更 `/experiments/projects/:projectId/methods/:methodId/transitions/:transitionId/status` → `/experiments/projects/:projectId/methods/:methodId/tasks/:taskNo/transitions/:transitionId/status` | | | - リクエストボディからパスパラメータ化した項目を削除 | | 2.0.0 | 2026-02-17 | 大規模仕様変更 | Cline | | | - テーブル変更: experiment_records → template_data | | | - パラメータ変更: taskId (UUID) → taskNo (integer) | | | - パラメータ追加: placeId（必須） | | | - 一括保存対応: records配列による複数データ保存 | | | - 一意キー変更: (org_id, proj_id, method_id, task_no, place_id) | | | - autoProgressUpdate機能削除 | | | - transitionId削除（不要） | | | - All or Nothing方式のトランザクション保証 | | 1.0.0 | 2026-01-27 | 初版作成 | Cline | | | - 実験データ記録API 2エンドポイントの設計書作成 | | | - 実装未着手 | |

注意: この設計書は実装済みAPIの仕様を記述したものです。仕様変更時はソースコード（ `experimentController.ts` 、 `experimentDataService.ts` ）と合わせて本ドキュメントも更新してください。